



2025

**Compiladores e Intérpretes
Generación de Código Intermedio y
Assembler**

RV

RouVel
COMPILER

Grupo 9

Iñaki Roumec : iroumec@alumnos.exa.unicen.edu.ar

Ulises Velis: uvelis@alumnos.exa.unicen.edu.ar

Profesor Asignado

Mailén Gonzalez: mgonzalez@intia.exa.unicen.edu.ar

Introducción.....	4
Temas Particulares.....	5
Tema 2.....	5
Tema 5.....	5
Tema 7.....	5
Tema 14.....	5
Tema 17.....	6
Tema 22.....	6
Tema 26.....	6
Tema 27.....	7
Tema 30.....	7
Tema 33.....	7
Controles en tiempo de Ejecución.....	8
Control A.....	8
Control B.....	8
Control F.....	8
Generación de Código Intermedio.....	9
Estructura de Almacenamiento.....	9
Uso de Notación Posicional.....	9
Algoritmos de Bifurcación.....	10
Selecciones.....	10
Iteraciones.....	11
Nuevos Errores Considerados.....	12
Otras Cuestiones.....	12
Funciones.....	12
Generación de Código Assembler.....	13
Mecanismos Utilizados.....	13
Generación del Código.....	13
Operaciones Aritméticas.....	15
Etiquetas.....	15
Modificaciones de las Etapas Anteriores.....	17
Tabla de Símbolos.....	17
Retorno.....	17
Asignaciones Múltiples.....	18
Resolución de los Temas Particulares.....	19
Tema 2.....	19
Tema 5.....	19
Tema 7.....	19
Tema 14.....	19
Tema 17.....	19
Tema 22.....	20
Tema 26.....	20
Tema 27.....	20



Tema 30.....	20
Tema 33.....	20
Salida del Programa.....	21
Experiencia.....	23
Conclusiones.....	23

Introducción

Como trabajo práctico de la cursada de la materia Compiladores e Intérpretes, se busca realizar un compilador, programando y desarrollando a fondo cada una de las siguientes etapas:

- Análisis léxico
- Análisis sintáctico
- Análisis semántico
- Generación de código intermedio
- Generación de código de salida
- Optimizaciones

Basadas en líneas generales en el siguiente gráfico proporcionado por la cátedra.

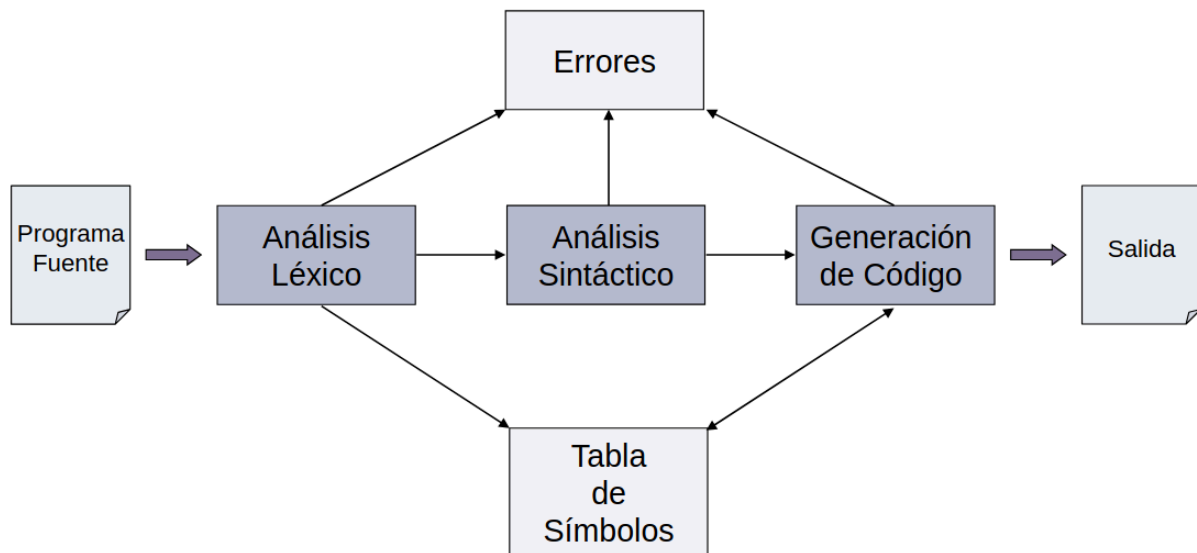


Imagen 1: etapas de un compilador.

En el informe anterior, se abordaron las etapas del análisis léxico y el análisis sintáctico. En este, se detalla el desarrollo e implementación del **análisis semántico**, la **generación de código intermedio** y la **generación de código de salida**.

Temas Particulares

A continuación, se realiza un repaso de los temas asignados al grupo.

Tema 2

Enteros sin signo de 16 bits. Se deben incorporar al lenguaje constantes con valores entre 0 y $2^{16} - 1$. Estas se escriben como una secuencia de dígitos seguidos del sufijo **UI**. Adicionalmente, se debe incorporar a la lista de palabras reservadas la palabra **uint**.

Tema 5

Punto flotante de 32 bits. Se deben incorporar al lenguaje números reales con signo y parte exponencial. Estos únicamente podrán ser utilizados como constantes, es decir, no podrán declararse variables de este tipo. La parte exponencial puede estar ausente. Si está presente, el exponente comienza con la letra **F**, seguido obligatoriamente del signo del exponente y su valor. El punto es obligatorio. Solo una de las partes **puede** estar ausente en una misma constante. Es decir, si falta la parte entera, debe estar presente la decimal, y viceversa.

El rango admitido para los números de punto flotante es el intervalo (1.17549435F-38, 3.40282347F+38) para los positivos y (-3.40282347F+38, -1.17549435F-38) para los negativos, además del valor 0.0.

Tema 7

Cadenas de una línea. Se debe incorporar al lenguaje cadenas que no puedan ocupar más de una línea. Estas deben estar delimitadas por comillas dobles. Ejemplo: "Hola Mundo".

Tema 14

do while. Se debe incorporar al lenguaje la siguiente estructura: **do** **<bloque_de_sentencias_ejecutables>** **while (<condicion>);**, donde: **<condicion>** posee la misma definición que la condición de las sentencias IF; y **<bloque_de_sentencias_ejecutables>** podrá contener una sentencia, o un grupo de sentencias ejecutables delimitado por llaves.

El bloque de sentencias ejecutables se ejecutará mientras la condición sea verdadera. La evaluación de la condición se efectuará luego de la ejecución del bloque.

Como consecuencia de esta estructura, se debe incorporar a la lista de palabras reservadas del lenguaje las palabras **do** y **while**.

Tema 17

Asignaciones que pueden tener menor número de elementos del lado izquierdo. Se deben reconocer asignaciones múltiples, que permitan una lista de variables, separadas por coma (",") del lado izquierdo de la asignación, y una lista de constantes separadas por coma (",") del lado derecho. El operador utilizado para la asignación será "=" y el número de variable del lado izquierdo puede ser menor al de constantes en el lado derecho.

Se deberá generar código para asignar cada elemento de la derecha al correspondiente de la izquierda, en el orden en que se presenten. En caso que del lado izquierdo haya menos elementos que en el lado derecho, se descartarán las expresiones sobrantes del lado derecho, y se informará la situación con un *warning*. En caso que del lado izquierdo haya más elementos que del lado derecho, se informará como error.

Se deberá chequear la compatibilidad de tipos en cada asignación individual, en forma independiente.

Tema 22

Prefijado obligatorio. Se debe incorporar la posibilidad de utilizar, en cualquier lugar donde pueda participar una variable, un prefijado que indique la unidad a la que pertenece. Este debe escribirse como un identificador seguido de un "." precediendo al nombre de la variable:

ID.ID.

Este prefijado se utilizará para indicar la unidad a la que pertenece la variable, en caso de que no esté declarada localmente. Este es obligatorio para variables no declaradas localmente. Por lo tanto:

- Si una variable no contiene el prefijado, se asume que es local, y debe buscarse sólo en el ámbito actual
- Si una variable tiene prefijado, se debe buscar en el ámbito indicado por el prefijado, chequeando su visibilidad mediante *name mangling*.

Tema 26

Copia-Valor-Resultado. Se debe agregar a la lista de palabras reservadas la palabra **cvr**. En la declaración de la función, antes de cada parámetro formal, se **podrá** indicar, mediante una palabra reservada, la semántica del pasaje de ese parámetro. Los casos posibles son:

- **<parametro_formal> <tipo> ID**, donde se usa la semántica por defecto: copia-valor.
- **<parametro_formal> cvr <tipo> ID**, donde se usa la semántica de copia-valor-resultado.

Cuando se invoque una función, el pasaje de parámetros deberá efectuarse según lo consignado en la declaración.

Tema 27

Expresiones lambda en línea. Se debe incorporar al lenguaje la definición y uso de expresiones lambda que pueden ser usadas en línea. La sintaxis de esta es: **<parámetro><cuerpo><argumento>**.

Estas expresiones permiten un solo parámetro, que se escribirá entre paréntesis con la estructura **<tipo> ID**. El cuerpo será un bloque de sentencias ejecutables delimitado por llaves. El argumento podrá ser un identificador o una constante. Este debe ir entre paréntesis. Ejemplo: **(uint A) { if (A > 1UI) print ("¡Hola!"); } (3I)**.

Al detectar una expresión Lambda, con la estructura adecuada, se debe generar el código que implemente las acciones determinadas por dicha expresión. Para el ejemplo anterior, el código generado debe copiar el valor del parámetro real (3I) en el parámetro real (A), y ejecutar el cuerpo.

Tema 30

Conversiones implícitas. Se deben incorporar al compilador conversiones en forma implícita, cuando se intente utilizar una constante de punto flotante como operando de una expresión o comparación, o como parámetro real de una función.

Tema 33

Comentarios multilínea. Se debe incorporar al lenguaje la posibilidad de escribir comentarios que puedan ocupar más de una línea. Estos deben comenzar con **##** y terminar con **##**.

Controles en tiempo de Ejecución

A continuación, se describen los controles en tiempo de ejecución asignados al grupo:

Control A

División por cero para datos enteros y de punto flotante. El código generado debe chequear que el divisor sea diferente de cero antes de efectuar una división. Si eso sucede, se debe emitir un mensaje de error y terminar la ejecución.

Control B

Overflow en sumas de enteros. El código generado debe controlar el resultado de la suma para los tipos de datos enteros. Si este excede el rango del tipo del resultado, se debe emitir un mensaje de error y terminar la ejecución.

Control F

Resultados negativos en restas de enteros sin signo. El código generado debe controlar el resultado de la operación indicada. Este control se aplica a operaciones entre enteros sin signo. Si una resta entre datos de este tipo arroja un resultado negativo, se debe emitir un mensaje de error y terminar la ejecución.

Generación de Código Intermedio

El método de generación de código intermedio asignado al grupo es la **polaca inversa**. A continuación, se describe la estructura de almacenamiento utilizada, el uso de la notación posicional, los algoritmos de bifurcación planteados y otras cuestiones relevantes a la etapa.

Estructura de Almacenamiento

La estructura de almacenamiento utilizada para el guardado del código intermedio es una lista. Se eligió dicha estructura por ser simple y adecuarse correctamente a los requerimientos del método de generación de código intermedio.

A medida que se reducen las reglas de la gramática, dentro de las acciones semánticas, se realiza el agregado de polacas a la lista. No obstante, no todos los agregados ocurren instantáneamente luego de una reducción, como se describe en secciones posteriores.

Uso de Notación Posicional

La notación posicional en la gramática tiene dos fines. El primero es el de acceder al lexema. Esto es útil para realizar el añadido de una polaca ante la detección de un operando u operador, como se ve en el siguiente código simplificado:

```
factor
    : constante
      { polacas.añadir($1); }
    ...
```

El acceder al lexema también sirve para detectar si una variable se halla declarada en el ámbito (en caso de no especificarse el prefijado), adjuntándole el ámbito mediante *name mangling* y verificando su aparición en la tabla de símbolos.

```
variable
    : ID
      {
        if (!tablaDeSimbolos.existeEntrada(añadirÁmbito($1)))
          notificarError("Variable %s no declarada.", $1);
      }
```

El segundo fin es el de facilitar los controles semánticos. Por ejemplo, en la lista de variables, en la asignación múltiple, es posible encontrar el siguiente fragmento:

list_of_variables

```
...
| variable ',' list_of_variables
  { $$ = $1 + ',' + $3; }
...
```

El objetivo detrás de este es que, al reducir por el no terminal correspondiente a la asignación múltiple, **multiple_assignment**, se tenga acceso, en formato de *string*, a la lista de variables. De esta forma, puede realizarse la separación de las variables, utilizando la coma como delimitador, y realizar el chequeo semántico de que el número de variables del lado izquierdo de la asignación no supere el número de constantes en el lado derecho de esta.

Algoritmos de Bifurcación

Hubo dos formas de tratar los tipos de bifurcaciones: una utilizada para las selecciones y otra utilizada para las iteraciones. A continuación, se describe en su correspondiente apartado cada una de ellas, junto a un pseudocódigo que permite entender el funcionamiento.

Selecciones

Al momento de que el lenguaje reconoce un *if*, no sabe cuál será la dirección de salto en caso de que la condición de evaluación no se cumpla. Por consiguiente, no puede establecer aún un número de polaca al cual saltar. La solución a este problema, como se vio en la teoría, es dejar la polaca incompleta y rellenarla cuando se cuenta con la información suficiente.

Para resolver esto, se utilizó un mecanismo al que se denominó “Mecanismo de Promesas”, el cual se implementó utilizando una pila: la **pila de promesas**. Al encontrar una declaración, se “promete” una polaca (agregando una polaca nula a la lista) y se agrega, luego, el tipo de bifurcación a la lista de polacas (en este caso, bifurcación por falso, la cual puede leerse en la polaca como FB, del inglés *false bifurcation*). La polaca prometida no tiene un valor asignado en ese momento. La promesa consiste en que, en un futuro, el valor de dicha polaca va a completarse.

```
function realizarPromesa() {
    polacas.añadir(null);
    pilaDePromesas.apilar(nroDePolacaActual);
}
```

De tener la selección una rama alternativa (*else*), se realiza otra promesa (nuevamente, añadiendo una polaca sin valor a la lista de polacas) y, posteriormente a ella, se añade la bifurcación por salto incondicional (la cual puede encontrarse, en la polaca, abreviada como UB, proveniente del inglés *unconditional bifurcation*). Luego de realizada esta promesa, se

cumple la anterior a ella, la mencionada en el párrafo previo, completando así el valor de la polaca precedente a la polaca FB.

```
function cumplirPromesa() {
    int promesa = pilaDePromesas.obtenerTope();
    polacas.reemplazar(promesa, nroDePolacaActual);
}
```

Por último, cuando la rama alternativa llega a su fin, se completa la promesa restante (la de bifurcación incondicional), rellenando así la polaca previa a la polaca UB.

En caso de no haber una rama alternativa, al llegar al fin de la selección, se completa la única promesa realizada: la promesa de bifurcación por falso.

El uso de una pila permite el anidamiento de múltiples *if-else*, colocando adecuadamente el número de polaca, independientemente del nivel de anidamiento.

Iteraciones

Para la iteración existente en el lenguaje, el *do-while*, se utiliza también una pila: la **pila de puntos de iteración**. Mediante esta, al final del cuerpo del *do*, es posible determinar la polaca a la que se debe saltar para repetir el ciclo.

Cuando una iteración da comienzo, se apila su número de polaca en la pila de puntos de iteración.

```
function apilarPuntoDeIteración {
    pilaDePuntosIteración.apilar(numeroDePolacaActual);
}
```

Cuando esta termina (esto es, se lee el *while* junto a su condición), se toma el último punto de iteración y se añaden una polaca con el valor del punto de iteración y, otra con el valor TB, indicando una bifurcación por verdadero (TB proviene del inglés *true bifurcation*).

```
function conectarConUltimoPuntoDeIteración {
    polacas.añadir(pilaDePuntosDeIteracion.obtenerTope());
}
```

El uso de una pila permite, similar a las selecciones, el anidamiento de varios *do-while*.

Nuevos Errores Considerados

A continuación, se detallan los nuevos errores considerados en esta entrega. No todos ellos fueron solicitados por la cátedra, sino que algunos se plantearon ya que resultaban lógicos o ayudaban a impedir generar, en la siguiente etapa, código no funcional. Se subrayan los errores solicitados:

- Variables/funciones no declaradas en el ámbito (no visibles).
- Si el parámetro formal tiene una semántica de pasaje de parámetros por copia-valor-resultado, el argumento no puede ser un objeto no referenciable (constante, expresión, invocación a función...).
- Sentencia no alcanzable (debido a que la función tiene un retorno previamente).
- Overflow en suma de enteros, resultado negativo en suma de enteros y división por cero (considerados tanto en tiempo de *runtime* como compilación).
- *Overflow* en otras operaciones de enteros, como multiplicación.
- *Overflow* en suma, resta y multiplicación de flotantes.

Otras Cuestiones

Funciones

Se consideró adecuado, dado que en la invocación de una función se debe especificar el parámetro formal al que el parámetro real corresponde, permitir que el orden de los argumentos varíe. Para ello, fue necesario plantear una estructura (clase) encargada de llevar registro de los parámetros formales con los que cuenta una función.

Cuando se realiza una invocación a una función, se añaden los argumentos a dicha estructura. Al finalizar la invocación, esta última es la encargada de generar las polacas resultantes, reordenando los argumentos de acuerdo al orden de los parámetros formales.

Generación de Código Assembler

Para la generación del código *assembler*, se tomó la decisión de usar **WebAssembly** (WASM), esto debido a que, frente a la alternativa (*assembler* para Pentium de 32 bits), es más moderno, presenta una mayor portabilidad y su uso va en crecimiento.

Previo a la explicación de los mecanismos utilizados, se considera importante realizar la aclaración de dos cuestiones:

- A diferencia de *assembler*, **WebAssembly no requiere de variables auxiliares para las conversiones**. Estas son automáticamente administradas por la herramienta. Como consecuencia, en la tabla de símbolos solamente se observan variables auxiliares correspondientes a las operaciones aritméticas.
- Tanto el generador de código *assembler* planteado por el grupo como WebAssembly utilizan una pila de operandos. Para romper la desambigüedad y aclarar de cuál se habla, se especificará: ***pila de operandos del generador*** y ***pila de operandos de WebAssembly***, respectivamente.

Con esto en cuenta, se prosigue con la explicación de los mecanismos.

Mecanismos Utilizados

Generación del Código

La generación del código *assembler* comienza tomando la salida de la anterior etapa: la lista de polacas. De no haber ningún error en la generación del código intermedio, esta última se comienza a recorrer de principio a fin. De hallarse un operando, este se apila en una pila de operandos. De hallarse un operador, se realizan sus acciones correspondientes.

En general, las acciones correspondientes a un operador consisten en la desaplicación de uno o dos operandos (aunque también existen operandos que no realizan ningún desapilado) y la generación del código correspondiente a su acción. Únicamente los operadores son responsables de generar código.

Todos los operadores comparten un repositorio en común: la clase **CodeRepository**. El agregado de código, la apertura de bloques y demás acciones la realizan comunicándose con dicha clase. Esta se encarga de ordenar el código, identificar el ámbito actual y preparar la salida a WebAssembly.

A continuación, se describen brevemente los operadores que el generador de código *assembler* reconoce:

- **+, -, * y /**. Estos cuatro operadores corresponden a las operaciones aritméticas. Se describen en la siguiente sección.

- **==, >=, <=, <, > y !=**. Son los operadores de comparación. Al encontrar uno de ellos, se desapilan dos operandos de la pila de operandos, se genera el código correspondiente al apilado de cada uno de los operandos en la pila de operandos de *WebAssembly* (haciendo uso de instrucciones como `local.get`, `i32.const` y `f32.const`, junto a sus conversiones) y se añade la instrucción de comparación adecuada al operador (por ejemplo, `i32.eq` para el operador `==`).
- **:=**. Representa a la asignación. Desapila dos operandos de la pila de operandos del generador, siendo el primer operando desapilado el valor a asignar y el segundo, la variable a la que se le asigna dicho valor. Posteriormente, genera el código correspondiente a la conversión del valor a asignar (en caso de ser una constante flotante) y su asignación a la variable de la izquierda.
- **->**. Indica el pasaje de un parámetro real a una invocación de función. Desapila dos operandos de la pila de operandos del generador. El primer operando que desapila es el parámetro formal y el segundo, el argumento o parámetro real. El código generado por el operador consiste en la carga de los operandos desapilados en la pila de operandos de *WebAssembly* (junto a las conversiones requeridas de ser el argumento un flotante).
- **call**. Indica la invocación de la función. Siempre aparece luego del pasaje de argumentos. Su código generado consiste en el pasaje de las variables declaradas hasta el momento de la declaración de la función invocada (de esta forma, solo se pasan como parámetro las variables de otros ámbitos que la función puede llegar a referenciar dentro de su cuerpo o que pueden llegar a referenciar otras funciones anidadas dentro de ella), el pasaje de las variables locales no auxiliares, la invocación de la función correspondiente, el guardado en una variable auxiliar del valor retornado por la función y la lectura de los valores de las variables locales y de otros ámbitos que se le pasaron como parámetro. Para conocer cuál es la función invocada, desapila un operando de la lista de operandos del generador.
- **<-**. Indica la lectura del resultado de los parámetros formales de la función invocada anteriormente. Hay uno de estos operadores por cada parámetro formal con un modelo semántico de pasaje de parámetro de copia-valor-resultado. El código que genera consiste en la asignación de los valores en la pila de operandos a cada uno de los argumentos.
- **print**. Indica la impresión de una expresión. Desapila de la pila de operandos del generador un operando. De acuerdo al tipo de operando (`UINT`, `String` o `Float`), genera el código correspondiente a la importación de la función *JavaScript* que realiza la impresión de ese tipo de dato en particular, además de la propia llamada a la función.
- **return**. Indica el retorno de una función. Desapila de la pila de operandos del generador un operando. El código generado consiste en la apilación de dicho operando en la pila de operandos de *WebAssembly* y el agregado de la operación *assembler return*.

Existen otros operadores definidos en el generador: estos son los relacionados a las etiquetas. Se describen en su sección correspondiente.

Operaciones Aritméticas

Los operadores correspondientes a las operaciones aritméticas siguen un mismo patrón:

- Desapilan dos operandos de la pila de operandos.
- De ser estas constantes, se aplica la optimización por **reducción simple**, detectando, en esta, posibles *overflows*, enteros negativos y divisiones por cero. En caso de detectar alguno de los anteriores casos, se muestra un error de compilación y se detiene la generación del código *assembler*. Por último, el resultado de la reducción se apila nuevamente en la pila de operandos.
- De ser al menos uno de los operandos una variable (la cual es obligatoriamente de tipo entero dadas las restricciones del lenguaje), se genera el código *assembler* que realiza la operación y efectúa los controles previos (como, por ejemplo, el chequeo de que el dividendo no sea cero en una división) y los posteriores a la operación (como, por ejemplo, el *overflow* de enteros y el resultado negativo). De ser uno de los operandos una constante flotante, se genera, además, el código que realiza su conversión a entero.

En este último caso, de ser al menos uno de los operandos una variable, el resultado de la operación se almacena en una **variable auxiliar**, la cual es posteriormente agregada a la tabla de símbolos. Dicha variable auxiliar, luego de haberse generado todo el código correspondiente a la operación, se agrega nuevamente a la pila de operandos del generador para ser utilizada como operando en otra operación.

Etiquetas

A continuación, se describirán las etiquetas, las cuales funcionan como operadores en el contexto del generador de código *assembler*. Estas tienen el objetivo de detectar y generar el código correspondiente a la declaración de funciones, *loops*, selecciones y *lambdas*.

- **open-function y close-function**. Colocadas, respectivamente, al inicio y al final de la declaración de la función. El código generado por ellas consiste en la creación y cierre, respectivamente, de la estructura de la función en *WebAssembly*, lo que comprende: la creación del bloque, el cálculo de la cantidad de parámetros y resultados (retornos) y su cierre. El cuerpo del bloque es rellenado con el código que se genere entre las etiquetas, siempre y cuando esté código no corresponda a una función con nombre o a una *lambda*. En estos últimos casos, se genera nuevamente otro bloque y, al cerrarse, se continúa con el abierto anteriormente.
- **open-lambda y close-lambda**. Funcionan de forma similar a las anteriores dos etiquetas, respectivamente, con la diferencia de que **open-lambda** no añade a la función un retorno fijo (puesto a que, a diferencia de las funciones con nombre, no

requiere de un retorno) y **close-lambda** no añade la sentencia unreachable al código generado (lo cual sí hace **close-function**, ya que así lo solicita WebAssembly).

- **open-selection, close-selection, FB y UB**. Estos cuatro operadores se utilizan en la generación del código de las selecciones:
 - **open-selection**. Realiza la apertura del bloque de la selección.
 - **FB**. Establece el cierre de la condición del IF. Genera las instrucciones de salto al final del bloque de la selección, en caso de ser la condición verdadera, o, en su defecto, al bloque *else* (de haberlo).
 - **UB**. Añade un salto al final de la selección y cierra el bloque correspondiente a la primera rama. Solo aparece si la selección cuenta con una rama *else*.
 - **close-selection**. Genera el código correspondiente al cierre de la selección, lo que incluye el cierre del bloque *else*.
- **open-loop, TB y close-loop**. Estos operadores se utilizan en la generación del código de las iteraciones.
 - **open-loop**. Realiza la apertura del bloque de la iteración. Desapila un operando de la pila de operandos del generador para formar la etiqueta del bloque.
 - **TB**. Genera el código correspondiente al salto al inicio del bloque que contiene la iteración (el cual se ejecuta cuando la condición es verdadera) y del salto al final de este (el cual se ejecuta en caso de ser la condición falsa).
 - **close-loop**. Realiza el cierre del bloque, manejando temas como la indentación y la colocación de la cantidad adecuada de paréntesis.

Modificaciones de las Etapas Anteriores

Tabla de Símbolos

La tabla de símbolos pasó de implementarse mediante un `HashMap` a implementarse mediante un `LinkedHashMap`. De esta forma, las entradas están ordenadas de acuerdo al orden de agregado. Esto es útil para, por ejemplo, conocer el orden de los parámetros formales en una función y reordenar los argumentos de acuerdo a estos.

Retorno

Algo que no se consideró en la etapa anterior es la prohibición de la sentencia *return* fuera del cuerpo de funciones. En esta entrega, se incorporó dicha restricción.

El chequeo no se planteó de forma sintáctica, sino semántica. Ante la aparición de una sentencia de retorno, declaración de función o selección, se notifica a una clase `ReturnsController` dicho evento. Esta lleva un registro de la cantidad de retornos encontrados, la cantidad de retornos necesarios, el nivel de profundidad actual, entre otras cuestiones. De esta forma, se permiten sentencias como la siguiente:

```
uint FUNCTION(...) {  
  
    if (...) {  
        ...  
        return(...);  
    } else {  
  
        if (...) {  
            return(...);  
        } else {  
            return(...);  
        } endif;  
    } endif;  
}
```

Una función de la anterior forma se considera totalmente válida en el lenguaje, puesto a que siempre tiene un punto de salida asegurado. `ReturnsController` permite que una función no requiera de un retorno explícito en su final si esta no lo necesita. Si bien esto no fue solicitado por la cátedra, no se consideró adecuada la alternativa de realizar el chequeo sintácticamente, obligando a colocar siempre un retorno al final de la función.

Adicionalmente, el algoritmo permite detectar, levantando una *warning*, sentencias no alcanzables. Estas, debido a que nunca se ejecutan, no son añadidas a la polaca y, por consiguiente, su código *assembler* tampoco se genera.

Asignaciones Múltiples

En la entrega anterior, el chequeo de que el número de variables no pueda exceder el número de constantes se realizaba mediante reglas de la gramática. No obstante, al momento de construir la generación del código intermedio, se percató de que dicha forma, si bien puede resultar más eficiente, dificulta altamente la construcción de dicho código. Por consiguiente, se redefinieron las reglas de la estructura, convirtiendo el chequeo sintáctico a un chequeo semántico y más arraigado al lenguaje de programación en el que el compilador se escribe.

El chequeo actualmente se realiza tomando la cadena de caracteres que representa la asignación múltiple. Posteriormente, se utiliza el operador de la asignación múltiple como delimitador para separar las variables de las constantes. Seguido a eso, se utiliza la coma como delimitador de las variables y constantes. De esta manera, es posible contar la cantidad de variables y la cantidad de constantes fácilmente. Adicionalmente, y la razón principal por la que se evaluó esta forma de abordar el problema, esto permite generar fácilmente el código intermedio correspondiente a la asignación, recorriendo a la par la lista de variables y la lista de constantes y añadiendo, en cada recorrido, la dupla en forma de una asignación individual.

Resolución de los Temas Particulares

A continuación, se mencionan los temas asignados al grupo, junto a su forma de resolución durante todas las etapas del compilador:

Tema 2

Enteros sin signo de 16 bits. El tema fue resuelto en el análisis léxico, agregando a la lista de palabras reservadas la palabra `uint` y añadiendo a la máquina de estados la detección de dichas constantes. Adicionalmente, se añadieron las acciones semánticas correspondientes para la comprobación de que su valor se halle en el rango.

Tema 5

Punto flotante de 32 bits. Similar a la resolución anterior, el tema fue resuelto principalmente en el análisis léxico, mediante la modificación de la máquina de estados para la detección de dichas constantes y el añadido de acciones semánticas para el chequeo de los rangos.

No obstante, a diferencia del anterior tema, en este fue necesario añadir adicionalmente una acción semántica en la gramática. Esto con la finalidad de detectar el signo “-” como parte de la constante e identificar así correctamente constantes negativas.

Tema 7

Cadenas de una línea. Este tema fue resuelto completamente en el análisis léxico al momento de definición de la máquina de estados.

Tema 14

do while. Este tema fue resuelto en varias etapas. En el análisis léxico, se añadieron las palabras reservadas correspondientes: `do` y `while`. En el análisis sintáctico, en la gramática, se definieron las reglas que permitieran validar la estructura. En la generación de código intermedio, en la polaca, se plantearon los mecanismos de bifurcación necesarios para su funcionamiento. Por último, en la generación de código *assembler*, se definieron las instrucciones que permitan la ejecución de la estructura mediante la condición sea verdadera.

Tema 17

Asignaciones que pueden tener menor número de elementos del lado izquierdo. Su resolución se dividió en varias etapas. En el análisis léxico, se contempló en la máquina de estados el operador de la asignación múltiple “=” . En el análisis sintáctico, se definieron las reglas que permitieran detectar la estructura. En la generación de código intermedio, se añadieron las acciones semánticas correspondientes al agregado de cada par de la asignación a la lista de polacas como una asignación simple. Por último, en la generación de código *assembler*, se

planteó la generación de las instrucciones correspondientes a la asignación, al igual que sus conversiones.

Tema 22

Prefijado obligatorio. El tema se resolvió en: el análisis léxico, mediante la incorporación del punto a los literales reconocidos por la máquina de estados; el análisis sintáctico, mediante la definición de la estructura **ID.ID**; y el análisis semántico, mediante el agregado de acciones semánticas que permitan, utilizando *name mangling*, corroborar si la variable existe en el ámbito indicado por el prefijado.

Tema 26

Copia-Valor-Resultado. En el análisis léxico, se añadió la palabra **cvr** a la lista de palabras reservadas. En el análisis sintáctico, se añadieron a la gramática las reglas necesarias para reconocer ambos tipos de estructuras: el parámetro formal con semántica por defecto (copia-valor) y el parámetro formal con semántica explícita (copia-valor-resultado). En la generación de código intermedio se añadieron a la polaca los elementos correspondientes a la copia del parámetro formal en el parámetro real (para el pasaje por copia-valor-resultado). Por último, en la generación del código *assembler*, se generó el código que comprenda la lógica de asignación adecuada.

Tema 27

Expresiones lambda en línea. En el análisis sintáctico, en la gramática, se definió la estructura correspondiente a las expresiones. En la generación de código intermedio, se definió el agregado en la polaca del parámetro, el cuerpo y el argumento de la expresión, y la operación de copia de este último en la polaca. Por último, en el código *assembler*, se realizó la generación de su código correspondiente mediante un tratamiento similar al de las funciones, como se describió en secciones anteriores.

Tema 30

Conversiones implícitas. Este tema se resolvió completamente en la generación de código *assembler*, añadiendo al código generado las conversiones requeridas de las constantes de punto flotante cuando aparecen como operando de una expresión o comparación, o como parámetro real de una función.

Tema 33

Comentarios multilinea. El tema se resolvió completamente en el análisis léxico, al momento de definir la máquina de estados del lenguaje.

Salida del Programa

La salida del programa se conforma de la siguiente manera:

```

=====
                                Resultados de la Compilación
                                Archivo: factorial.uki
=====

> Compilación Finalizada <
El programa tiene 17 líneas. Se detectaron 1 warnings y 0 errores.

> Warnings <
WARNING SINTÁCTICA: Línea 13: Sentencia 'return' no alcanzable. No se ejecutará.

=====

                                Polaca Inversa

                                Entering scope 'FACTORIAL'...

                                Entering scope 'FACT'...

1. FACT:FACTORIAL
2. open-function
3. open-selection
4. X:FACTORIAL:FACT
5. 1UI
6. ==

                                Entering 'if' body...

7. 13
8. FB
9. 1UI
10. return
11. 36
12. UB

                                Entering 'else' body...

13. open-selection
14. X:FACTORIAL:FACT
15. 0UI

16. ==

                                Entering 'if' body...

17. 23
18. FB
19. 1UI
20. return
21. 35
22. UB

                                Entering 'else' body...

23. X:FACTORIAL:FACT
24. X:FACTORIAL:FACT
25. 1UI
26. -
27. X:FACTORIAL:FACT
28. ->
29. FACT:FACTORIAL
30. call
31. read-return
32. *
33. return
34. close-selection

                                Leaving 'if-else' body...

35. close-selection

                                Leaving 'if-else' body...

```

```

=====
36. 0UI
37. FACT:FACTORIAL
38. close-function
=====
Leaving scope 'FACT'...
=====
39. 5UI
40. X:FACTORIAL:FACT
41. ->
42. FACT:FACTORIAL
43. call
44. read-return
45. print
=====
Leaving scope 'FACTORIAL'...
=====

=====
                          Tabla de Símbolos
=====
Lexema          | Valor          | Tipo          | Categoría          | Referencias
=====
FACTORIAL       |                |               | Program Name      | 2
=====
FACT:FACTORIAL  |                | Uint          | Function           | 3
=====
X:FACTORIAL:FACT |                | Uint          | CV Parameter       | 7
=====
1UI             | 1              | Uint          | Constant           | 4
=====
0UI             | 0              | Uint          | Constant           | 2
=====
5UI             | 5              | Uint          | Constant           | 1
=====
aux0:FACTORIAL:FACT |                | Uint          | Auxiliar Variable  | 1
=====
aux1:FACTORIAL:FACT |                | Uint          | Auxiliar Variable  | 1
=====
aux2:FACTORIAL:FACT |                | Uint          | Auxiliar Variable  | 1
=====
aux3:FACTORIAL  |                | Uint          | Auxiliar Variable  | 1
=====

=====
                          Archivo .wat generado: outputs/wat/factorial.wat
=====
                          Archivo .wasm generado: outputs/wasm/factorial.wasm
=====

=====
                          Resultados de la compilación guardados en: outputs/results/factorial.txt
=====

```

Imagen 1: ejemplo de salida del programa.

En ella es posible observar, al comienzo, un resumen del número de líneas en el programa y la cantidad de *warnings* y errores detectados. Posteriormente, de haber errores o *warnings*, estos son mostrados. A continuación de estos, se presenta la lista de polacas generadas. Posteriormente, es posible observar la tabla de símbolos. Esta corresponde a la tabla resultante luego de realizada la generación de código intermedio. Por último, se muestran las rutas en las que se guardaron los archivos `.wat` y `.wasm` (este último solo si se cumple que el sistema tenga la herramienta `wat2wasm` instalada) y la ubicación en la que se guardaron de forma persistente los resultados de la compilación. En caso de ocurrir un error, se presenta el siguiente mensaje y no se incluyen ni la tabla de símbolos ni la polaca.

```

=====
El código contiene errores, por lo que no fue posible generar un código assembler.
=====

```

Imagen 2: ejemplo de mensaje de error si el programa falla.

Experiencia

El desarrollo de estas etapas fue intuitivo y entretenido.

Las mayores frustraciones aparecieron en la generación del código *assembler* de las selecciones y en el algoritmo de detección de retornos. Sin embargo, estos desafíos fueron superados eventualmente.

Durante la finalización del proyecto, el poder observar el desarrollo y funcionamiento de cada una de las etapas reflejado en un programa funcional fue una experiencia gratificante.

Conclusiones

El diseño de un compilador comprende una constante toma de decisiones en cada una de sus etapas que repercutirá directamente en la facilidad de diseño e implementación de la siguiente, además de en otras cuestiones relacionadas a la calidad del código. No es una tarea fácil ni corta.

Durante el desarrollo del compilador, no solo se asentaron los conocimientos vistos en las clases teóricas y se aprendió a utilizar nuevas herramientas, como Byacc/J y WebAssembly, sino que también se refrescaron conocimientos de otras materias (como Lenguajes Formales y Autómatas, Programación Orientada a Objetos, Diseño de Sistemas de Software y Programación Web). Adicionalmente, se destaca el trabajo colaborativo como parte fundamental del proyecto, puesto a que las discusiones fueron el pilar de cada una de las ideas que se ven reflejadas en este.